CECS 440 – Lab 8

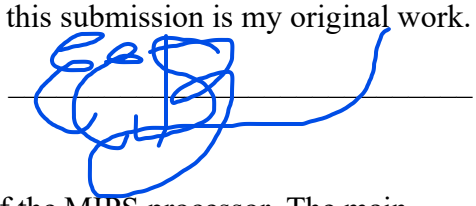
Memory Access Stage

Due date: 11/11/2025

Name: Eric Santana

ID: 015107467

I certify that this submission is my original work.



Goal

The goal of this lab was to build the Memory Access stage of the MIPS processor. The main objectives were to create the data memory module, handle memory read and write operations, generate the PCSrcM signal for branching, and pass the correct control and data signals to the Write Back stage. The design was then tested in the simulation to confirm that memory access and branch logic worked correctly.

Steps

1. Opened Vivado and created a new project for lab 8.
2. Create the dmem module and MEM_Stage module.
3. Wrote and completed the Verilog code for each module according to the lab instructions.
4. Created a testbench to simulate the functionality of the design.
5. Ran the simulation and observed waveform and console output.

Results

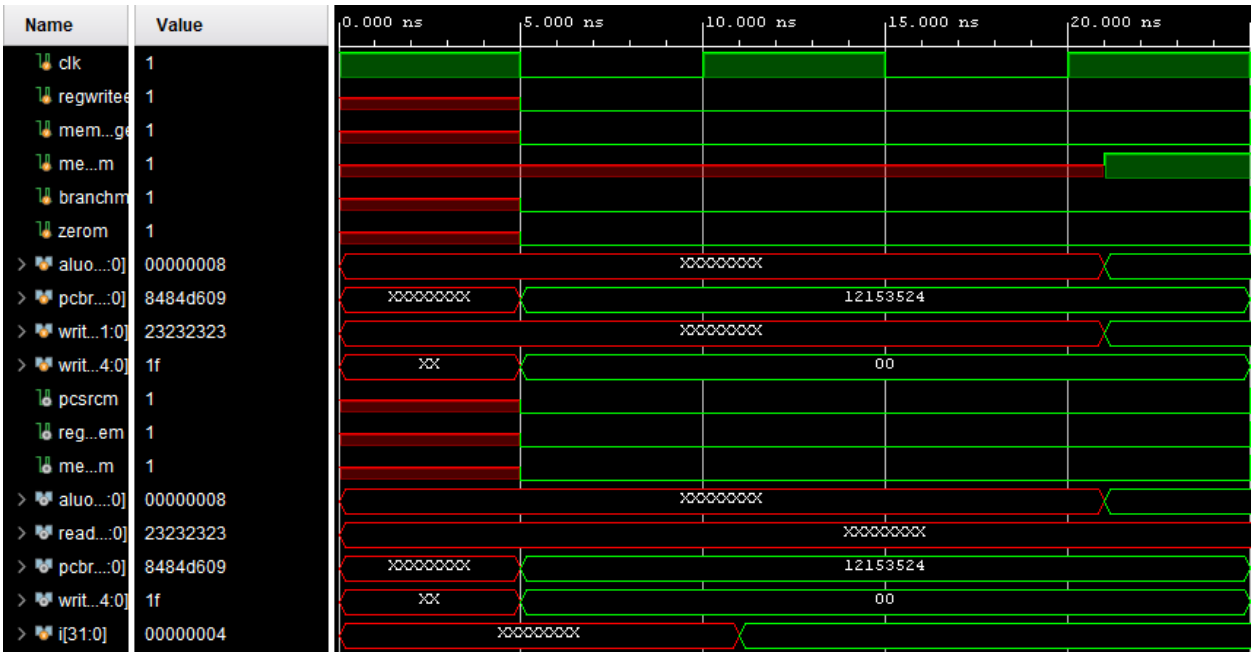
The simulation showed that the MEM stage worked as expected. Memory writes happened correctly when MemWriteM was set, and the data could be read back through ReadDataM. The PCSrcM signal was also generated correctly when both BranchM and ZeroM were 1. All passthrough signals, such as RegWriteM, MemtoRegM, ALUOutM, and WriteRegM, matched the inputs from the Execute stage. I originally ran into issues because of mismatched signal names, but after fixing those, the design simulated with the expected output.

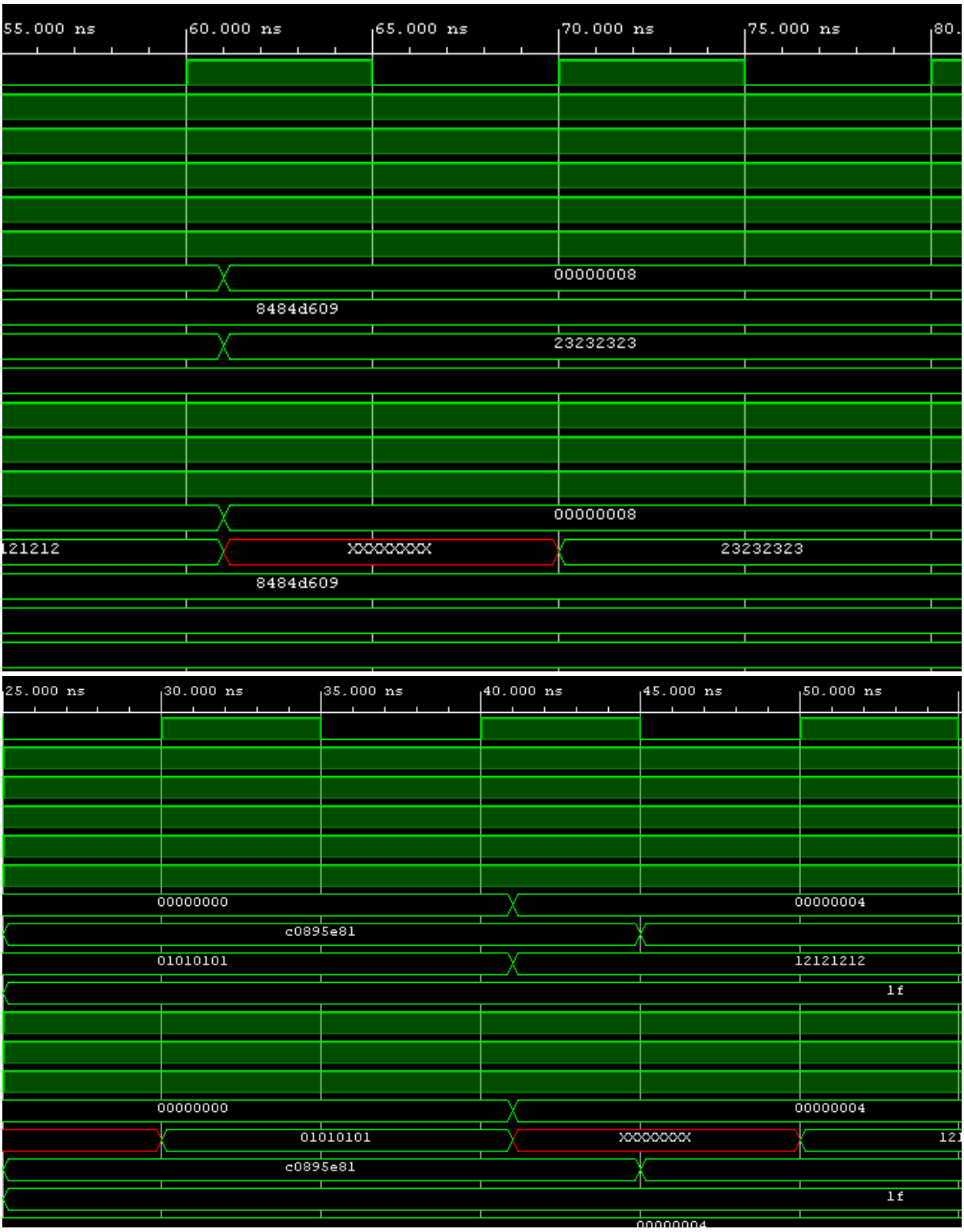
Conclusion

This lab helped me understand how the MEM stage fits into the MIPS datapath and how it handles memory operations and branching. It also showed how important it is for control and data signals to be passed correctly between pipeline stages. After debugging my signals and getting the simulation to run, I gained a better idea of how load, store, and branch instructions work in hardware.

Appendix (1 out of 3)

Waveforms




55.000 ns
60.000 ns
65.000 ns
70.000 ns
75.000 ns
80.
00000008
8484d609
23232323
00000008
121212
XXXXXXXX
23232323
8484d609
25.000 ns
30.000 ns
35.000 ns
40.000 ns
45.000 ns
50.000 ns
00000000
00000004
c0895e81
01010101
12121212
1f
00000000
00000004
01010101
XXXXXXXX
c0895e81
1f

Appendix (2 out 3)

Console Output

```
# }
# run 1000ns
dmem[00000000] = xxxxxxxx
dmem[00000001] = xxxxxxxx
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = xxxxxxxx
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = 12121212
dmem[00000002] = xxxxxxxx
dmem[00000003] = xxxxxxxx
Control signals OK
dmem[00000000] = 01010101
dmem[00000001] = 12121212
dmem[00000002] = 23232323
dmem[00000003] = xxxxxxxx
$finish called at time : 81 ns : File "C:/Users/fallo/Desktop/CECS 440/Lab 8/Lab
INFO: [USF-XSim-96] XSim completed. Design snapshot 't_MEM_Stage_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:03 ; elapsed = 00:00:06 . Memory (MB):
```

Appendix (3 out 3)

Dmem.v

```
`timescale 1ns / 1ps

//Name: Eric Santana
//ID: 015107467
//Lab: 8
//Class: CECS 440
//CSULB

module dmem(clk, we, a, wd, rd);
    input clk, we;
    input [31:0] a, wd;
    output [31:0] rd;

    reg [31:0] RAM[63:0];


    assign rd = RAM[a[7:2]];


    always @(posedge clk)
        if (we) RAM[a[7:2]] <= wd;

endmodule
```

MEM_Stage.v

```
`timescale 1ns / 1ps
//Name: Eric Santana
//ID: 015107467
//Lab: 8
//Class: CECS 440
//CSULB

module MEM_Stage(
    input wire clk,
    input wire regwritee,
    input wire memtorege,
    input wire memwritem,
    input wire branchm,
    input wire zerom,
    input wire [31:0] aluoute,
    input wire [31:0] writedatam,
    input wire [4:0] writerege,
    output wire pcsrcm,
    input wire [31:0] pcbranche,
    output wire regwritem,
    output wire memtoregm,
    output wire [31:0] aluoutm,
    output wire [31:0] readdatam,
    output wire [4:0] writeregm,
    output wire [31:0] pcbranchm
);

    assign regwritem  = regwritee;
    assign memtoregm  = memtorege;


    assign pcsrcm     = branchm & zerom;

    assign aluoutm    = aluoute;
    assign writeregm  = writerege;


    assign pcbranchm  = pcbranche;


    dmem dmem(
        clk,
        memwritem,
        aluoute,
        writedatam,
        readdatam
    );

endmodule
```

T_MEM_Stage.v

```
`timescale 1ns / 1ps
//Name: Eric Santana
//ID: 015107467
//Lab: 8
//Class: CECS 440
//CSULB

module t_MEM_Stage();

    reg clk, regwritee, memtorege, memwritem, branchm, zerom;
    reg [31:0] aluoute, pcbranche, writedatam;
    reg [4:0] writerege;
    wire pcsrcm, regwritem, memtoregm;
    wire [31:0] aluoutm, readdatam, pcbranchm;
    wire [4:0] writeregm;
    integer i;


    MEM_Stage dut(
        clk,
        regwritee,
        memtorege,
        memwritem,
        branchm,
        zerom,
        aluoute,
        writedatam,
        writerege,
        pcsrcm,
        pcbranche,
        regwritem,
        memtoregm,
        aluoutm,
        readdatam,
        writeregm,
        pcbranchm
    );

    always #5 clk = ~clk;

    initial begin
        clk = 1;

        @(negedge clk) rndctrlsig(); showmem(); checksignals();
        memwritem = 1; aluoute = 32'h0; writedatam = 32'h01010101;

        @(negedge clk) rndctrlsig(); showmem(); checksignals();
        memwritem = 1; aluoute = 32'h4; writedatam = 32'h12121212;

        @(negedge clk) rndctrlsig(); showmem(); checksignals();
        memwritem = 1; aluoute = 32'h8; writedatam = 32'h23232323;

        #10 showmem();
        $finish;
    end

    task checksignals;
    begin @(posedge clk) #1
        if (regwritem != regwritee ||
            memtoregm != memtorege ||
            pcsrcm != (branchm & zerom) ||
            aluoutm != aluoute ||
            writeregm != writerege ||
            pcbranchm != pcbranche)
        begin
            $display("Test Failed: Control Signals faulty");
            $finish;
        end
        else begin
            $display("Control signals OK");
        end
    end
    endtask

    task rndctrlsig;
        {regwritee, memtorege, branchm, zerom, writerege, pcbranche} = $random;
    endtask

    task showmem;
    begin @(posedge clk) #1
        for (i = 0; i < 4; i = i + 1)
            $display("dmem[%h] = %h", i, dut.dmem.RAM[i]);
    end
    endtask

endmodule
```

Use of AI Tools:

ChatGPT Edu (GPT-5) was used to proofread the lab report for clarity and formatting.